

Assignment 8: Chihuahua vs Muffin Image Classification with PyTorch

Objective

Design, train, validate, and improve your own Convolutional Neural Network (CNN) using PyTorch on your local PC. The model must classify images into two classes: **Chihuahua** and **Muffin**. You will first create a BaselineCNN, then create a deeper ImprovedCNN and compare the validation results.

Important Rules

- The CNN model must be **designed by you**.
- Do not use pretrained models such as ResNet, VGG, MobileNet, EfficientNet, or similar networks.
- The **BaselineCNN** must contain at least **3 Conv2d layers, 3 MaxPool2d layers, and 3 Linear layers**.
- The **ImprovedCNN** must contain **more Conv2d layers** than the BaselineCNN.
- The ImprovedCNN must still contain at least 3 MaxPool2d layers and 3 Linear layers.
- Use `nn.CrossEntropyLoss()`. The final model output must be 2 raw logits; do not place Softmax inside the model.
- The validation set is for evaluation only. Do not move validation images into the training set.
- **Part B is mandatory even if the BaselineCNN already achieves more than 50% validation accuracy.**
- The 50% accuracy requirement applies to the final ImprovedCNN, not to the baseline stopping point.

0. Local PC Setup

Use Jupyter Notebook, JupyterLab, or VS Code. Python 3.10 or newer is recommended. A CUDA-capable NVIDIA GPU is optional; CPU training is allowed.

Windows example:

```
python -m venv .env
.venv\Scripts\activate
pip install -r Assignment_8_Local_PC_requirements.txt
jupyter notebook
```

macOS/Linux example:

```
python -m venv .env
source .venv/bin/activate
pip install -r Assignment_8_Local_PC_requirements.txt
jupyter notebook
```

Expected output: *Python/PyTorch version, CUDA availability, and either Device: cuda or Device: cpu.*

1. Prepare the Dataset

Suggested dataset (Kaggle):

<https://www.kaggle.com/datasets/samueltcortinhas/muffin-vs-chihuahua-image-classification>

Download and extract the images on your PC. Organize them into the following structure:

```
Assignment8/  
|-- Assignment_8_Chihuahua_Muffin_Local_PC.ipynb  
|-- dataset/  
|   |-- train/  
|   |   |-- chihuahua/  
|   |   |-- muffins/  
|   |-- val/  
|       |-- chihuahua/  
|       |-- muffins/  
|-- outputs/
```

- Keep the two classes reasonably balanced.
- Training images are used to update model weights.
- Validation images are used only to measure performance.
- If your dataset is stored elsewhere, change DATA_ROOT in the notebook.

Expected output: image counts for both classes and Dataset structure check: PASSED.

2. Load and Prepare the Images

- Use ImageFolder and DataLoader.
- Resize images to 128 x 128 pixels.
- Convert images to tensors and normalize them.
- Simple training augmentation such as horizontal flip and small rotation is allowed.
- Validation transforms must not contain random augmentation.
- The local-PC template uses num_workers=0 for better compatibility, especially on Windows.

Expected output: class names, class-to-index mapping, dataset sizes, a batch tensor shape, and a preview of training images.

3. Part A - Design Your Baseline CNN

You must write the `BaselineCNN` class yourself. The notebook does not provide the completed model.

```
Input Image
|
Conv2d -> ReLU -> MaxPool2d
|
Conv2d -> ReLU -> MaxPool2d
|
Conv2d -> ReLU -> MaxPool2d
|
Flatten (or Adaptive Pool + Flatten)
|
Linear -> activation
|
Linear -> activation
|
Linear -> 2 output logits
```

- At least 3 `nn.Conv2d` layers.
- At least 3 `nn.MaxPool2d` layers.
- At least 3 `nn.Linear` layers.
- You decide channel sizes, kernel sizes, hidden units, activations, `BatchNorm`, and `Dropout`.
- Final forward output shape must be `[batch_size, 2]`.
- Do not copy a pretrained network.

Expected output: the model prints successfully and the architecture checker reports `Architecture check: PASS`.

4. Train the Baseline Model

- Use `nn.CrossEntropyLoss()`.
- Choose an optimizer such as Adam or SGD.
- Start with at least 5 epochs.
- Record training loss, training accuracy, validation loss, and validation accuracy every epoch.
- Save or remember the best baseline validation accuracy.

Important: The baseline is only a benchmark. There is no pass/fail accuracy target for the baseline. If your `BaselineCNN` already achieves more than 50%, you must still continue to Part B.

Expected output: one line per epoch plus baseline loss and accuracy graphs.

5. Part B - Design a Deeper Improved CNN

Create a second class named ImprovedCNN. This part is mandatory.

- The ImprovedCNN must contain more Conv2d layers than the BaselineCNN.
- It must still contain at least 3 MaxPool2d layers and 3 Linear layers.
- You may change channels, kernel sizes, hidden units, activations, BatchNorm, Dropout, optimizer, learning rate, epochs, and training augmentation.
- Do not use the validation images for training.
- Do not use a pretrained model.

Goal: Train a deeper CNN and compare it with your baseline.

- The minimum final requirement is **ImprovedCNN validation accuracy > 50%**.
- You should try to achieve validation accuracy equal to or higher than the baseline.
- A deeper network is not guaranteed to beat a strong baseline. The ImprovedCNN does not have to exceed the baseline to make the assignment valid.
- However, the ImprovedCNN must be designed, trained, evaluated, and shown in your submission.

Expected output: Improved architecture check: PASS and best ImprovedCNN validation accuracy greater than 50%.

6. Compare Baseline and Improved Results

- Plot training and validation loss for both experiments.
- Plot training and validation accuracy.
- Print the best baseline validation accuracy.
- Print the best improved validation accuracy.
- Print the difference between the two validation results.
- Your final ImprovedCNN result must show validation accuracy > 50%.

Important: Do not stop after the baseline even if the baseline already achieves 50%, 80%, 90%, or higher.

7. Save and Reload the Final Model

Save the final ImprovedCNN model on your local PC in the outputs/ folder:

```
torch.save(improved_model.state_dict(),  
           './outputs/assignment8_best_model.pth')
```

Reload the weights into a new ImprovedCNN object and run validation inference.

Expected output: Model reload: SUCCESS and validation images showing true and predicted labels.

Submission Requirements

Submit exactly **two files** to Microsoft Teams.

1. Completed Code

StudentID_Assignment8.ipynb

- Must run on your local PC.
- Must contain your own completed BaselineCNN and ImprovedCNN classes.
- Do not submit a pretrained model solution.

2. Screenshot PDF

StudentID_Assignment8.pdf

Place the following screenshots in this order:

- Local PC environment output showing Python/PyTorch and Device.
- Dataset image counts and Dataset structure check: PASSED.
- Training image preview.
- Complete BaselineCNN code.
- Baseline model print and Architecture check: PASS.
- Baseline training output and best baseline validation accuracy.
- Baseline loss and accuracy graphs.
- Complete ImprovedCNN code.
- Improved model print and Improved architecture check: PASS.
- Improved training output.
- Improved loss and accuracy graphs.
- Comparison showing both baseline and improved best validation accuracy.
- Final result showing ImprovedCNN validation accuracy > 50%.
- Model reload: SUCCESS and validation prediction images.

No written discussion is required. Do not submit the image dataset unless the instructor asks for it.

Final Reminder: Baseline > 50% does not finish the assignment. You must still design, train, and evaluate the deeper ImprovedCNN.